

Принципы ООП

Исторически сложилось так, что ООП основывается на четырех принципах:

- абстракция
- инкапсуляция
- наследование
- полиморфизм



Иногда количество принципов сокращают до трех, опуская принцип абстракции.

Абстракция

Абстракция — это использование только тех характеристик объекта, которые с достаточной точностью представляют его в данной системе. Основная идея состоит в том, чтобы представить объект минимальным набором атрибутов и методов и при этом с достаточной точностью для решаемой задачи.

Абстракция является основой объектно-ориентированного программирования и позволяет работать с объектами, не вдаваясь в особенности их реализации. Пользователь типа не имеет прямого доступа к его реализации, но может работать с данными через предоставленный набор операций. Преимущество абстракции данных в разделении операций над данными и внутреннего представления этих данных, что позволяет изменять реализацию, не затрагивая пользователей данного типа.

Пример 1. Объекту класса Программист вряд ли понадобятся свойства "умение готовить еду" или "любимый цвет". Они не влияют на его особенности как программиста. А вот "основной язык программирования" и "рабочие навыки" — важные свойства, без которых программиста не опишешь.

Пример 2. Человек, управляющий скутером, знает, что при нажатии на гудок издается звук, но зачастую он не имеет представления о том, как на самом деле этот звук создается при нажатии на гудок.



Принцип абстракции позволяет нам скрывать детали и раскрывать только основные черты объекта.

Инкапсуляция

Инкапсуляция имеет два основных смысла. С одной стороны, она объединяет атрибуты и методы в одном объекте. С другой стороны, инкапсуляция обозначает сокрытие данных, то есть невозможность напрямую получить доступ к внутренней структуре объекта, так как это может быть небезопасно. Например, наполнить желудок едой можно напрямую, положив еду в желудок. Но это опасно. Поэтому прямой доступ к желудку закрыт. Чтобы наполнить его едой, нужно совершить ритуал через элемент интерфейса под названием рот.



Принцип инкапсуляции позволяет объектам содержать как свои данные, так и поведение, а также скрывать то, что ему потребуется, от внешнего программного кода.

Наследование

Наследование — способ создания класса на основе уже существующего, при котором **дочерний класс** заимствует атрибуты и методы **родительского класса**, а также добавляет собственные.

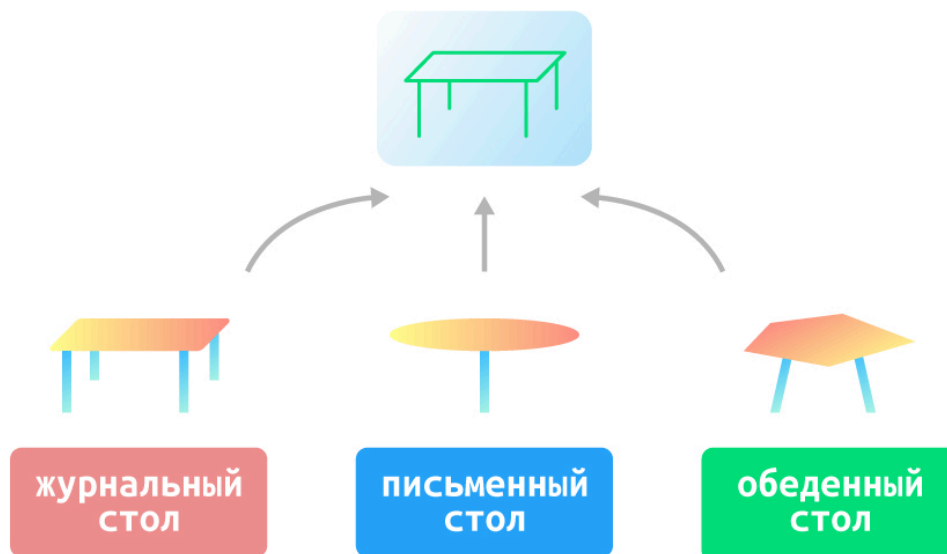


Дочерний класс часто называют **производным классом**, **наследником** или **потомком**, а родительский класс – **предком** или просто **родителем**.

Проведем аналогию с реальным миром. Если мы возьмем конкретный стол, то это объект, но не класс. А вот общее представление о столах, их назначении – это класс. Ему принадлежат все реальные объекты столов, какими бы они ни были. Класс столов дает общую характеристику всем столам в мире, описывая их общие свойства. Однако можно разделить все столы на письменные, обеденные и журнальные и для каждой группы создать свой класс, который будет наследником общего класса, но помимо этого будет вносить ряд своих особенностей.

Таким образом, общий класс будет родительским, а классы групп – дочерними. Дочерние классы наследуют особенности родительских, однако дополняют или в определенной степени модифицируют их характеристики. Когда мы создаем конкретный экземпляр стола, то должны выбрать, какому классу столов он будет принадлежать. Если он принадлежит классу журнальных столов, то получит все характеристики общего класса столов и класса журнальных столов. Но не особенности письменных и обеденных.

Стол



Наследование может быть одиночным, а может быть множественным. При множественном наследовании, у класса может быть более одного родителя. В этом случае класс наследует атрибуты и методы всех родительских классов. Достоинства такого подхода в большей гибкости. В то же время множественное наследование — потенциальный источник ошибок, которые могут возникнуть из-за наличия одинаковых имён атрибутов и методов в родительских классах.

Полиморфизм

Полиморфизм – это множество форм. Однако в понятиях ООП имеется в виду, скорее, обратное. Объекты разных классов, с разной внутренней реализацией, могут иметь одинаковые интерфейсы. Например, для чисел есть операция сложения, обозначаемая знаком `+`. Однако мы можем определить класс, объекты которого также будут поддерживать операцию, обозначаемую этим знаком. Но это вовсе не значит, что объекты должны быть числами и будет получаться какая-то сумма. Операция `+` для объектов нашего класса может значить что-то иное. Но интерфейс, в данном случае это знак `+`, у чисел и нашего класса будет одинаков. Полиморфность же проявляется во внутренней реализации и результате операции.

Мы уже сталкивались с полиморфизмом операции `+` в языке Python. Для чисел она обозначает сложение, а для строк – конкатенацию. Внутренняя реализация кода для этой операции у чисел отличается от реализации таковой для строк.



Полиморфизм позволяет выполнять одно действие разными способами, другими словами, он позволяет определять один интерфейс и иметь множество реализаций.

Примечания

Примечание 1. Наследование является механизмом повторного использования кода. Установка отношения наследования между классами порождает иерархию классов. Наследование полезно с точки зрения повторного использования кода. Однако при использовании наследования мы можем создавать сильно связанные иерархии. Если не быть осторожным, наследование может привести к огромной иерархической структуре классов, которую трудно понять и поддерживать.

When class Cat extends Table because it has four legs.

